Laboratory Work 3

Docker & Kubernetes | Reflection Report

Student	Course	Scope
Yevgenii Kurlykov	Distributed & Cloud Computing	Parts A, B and C

01 What is the fundamental difference between a container and a virtual machine?

A virtual machine runs a complete guest operating system (including its own kernel) on virtualized hardware managed by a hypervisor. A container isolates an application and its dependencies as operating-system processes while sharing the host kernel. As a result, containers typically have lower overhead, start faster and use fewer resources. Virtual machines offer stronger separation of operating systems; containers provide lightweight application-level isolation.

02 Why copy requirements.txt and install dependencies before copying application code?

Docker builds an image in layers and can reuse unchanged layers from its build cache. When the Dockerfile copies requirements.txt and executes pip install first, changing only app.py does not invalidate the dependency-installation layer. Docker can reuse the installed packages and rebuild only the later layers. This improves build speed, especially while the application code changes frequently. If requirements.txt changes, dependencies are installed again.

03 Why use a Deployment instead of a bare pod? What happens after deleting a pod?

A bare pod is not maintained by a workload controller: after an explicit deletion, no controller is responsible for recreating it. A Deployment manages a ReplicaSet and continuously reconciles the desired number of replicas with the actual state. With replicas: 3, deleting one managed pod causes Kubernetes to create a replacement so the desired count is restored. Other healthy pods can continue responding through the Service while the replacement starts. This is Kubernetes self-healing.

04 How does a Service find its pods, and why should replicated services be stateless?

A Kubernetes Service selects pods using labels. In our manifests, the selector app: my-api matches the labels on the Deployment's pod template. Kubernetes tracks matching endpoints and routes Service traffic to healthy, ready pods. A stateless API does not depend on local memory from a previous request, so any replica can handle the next request. Otherwise, replicas would need a shared session or data store to prevent inconsistent responses or lost state. The /time and /echo routes are stateless.

Submission note. Runtime evidence (build, running container, Compose, pods, distribution, deletion and scaling) is documented in the separately captured screenshots/ folder. These results must be verified on the student's own system.